

Northern University of Business and Technology Khulna

Department of Computer Science and Engineering

Project Title

MediLink – Smart Doctor Appointment & Healthcare Management System.

Submitted By

Name: Md. Ekramul Haque

ID: 11220320838

Name: Emam Mahadi Hasan

ID: 11220320832

Name: Lamia Ahmed Zim

ID: 11220320952

Semester: 8A

Submitted To

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and
Engineering , NUBTK

CSE 4104

Mobile Computing Lab

Team Name: CSE4204-T06

Team Leader: Md.Ekramul Haque

Department of Computer Science and Engineering

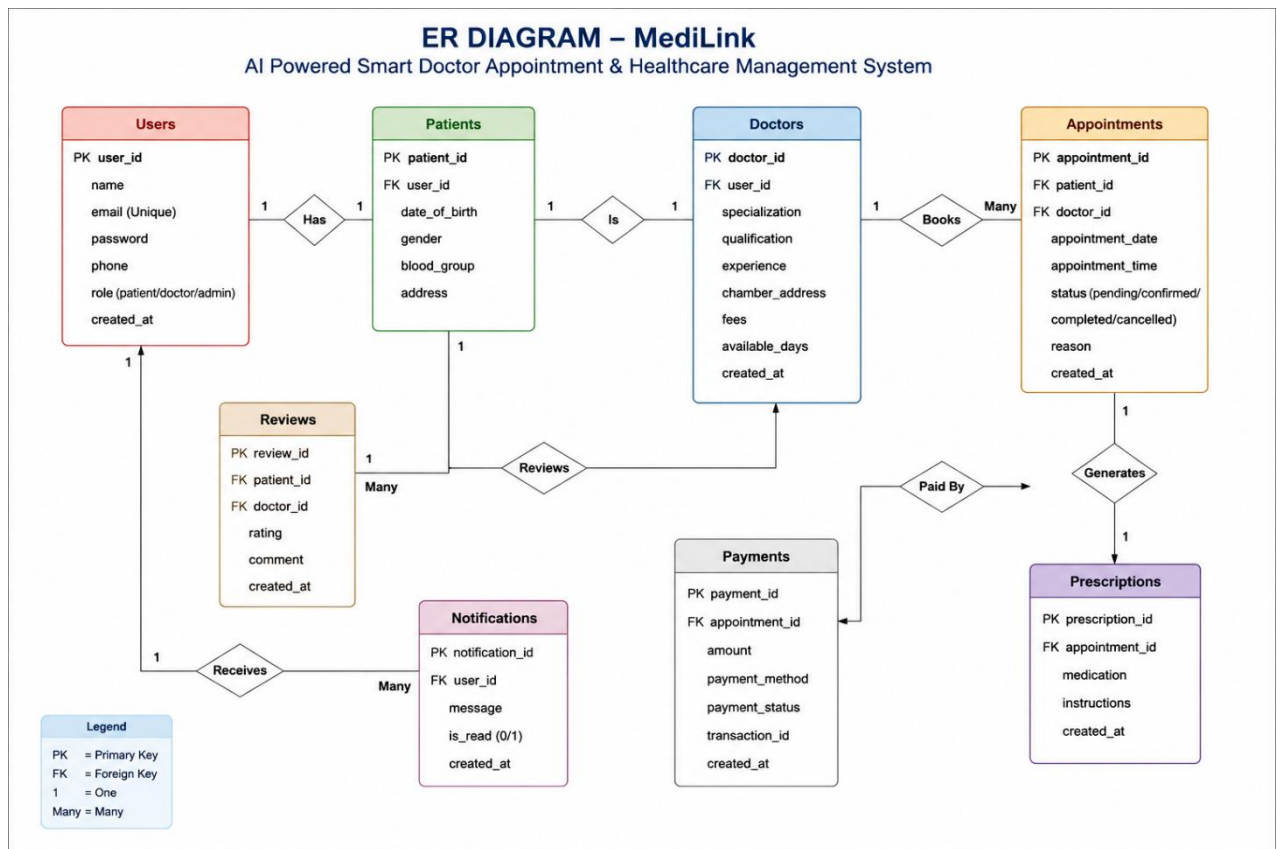
Northern University of Business and Technology Khulna

Khulna 9208, Bangladesh

29 June, 2026

Database Design and Architecture Description of MediLink

ER Diagram

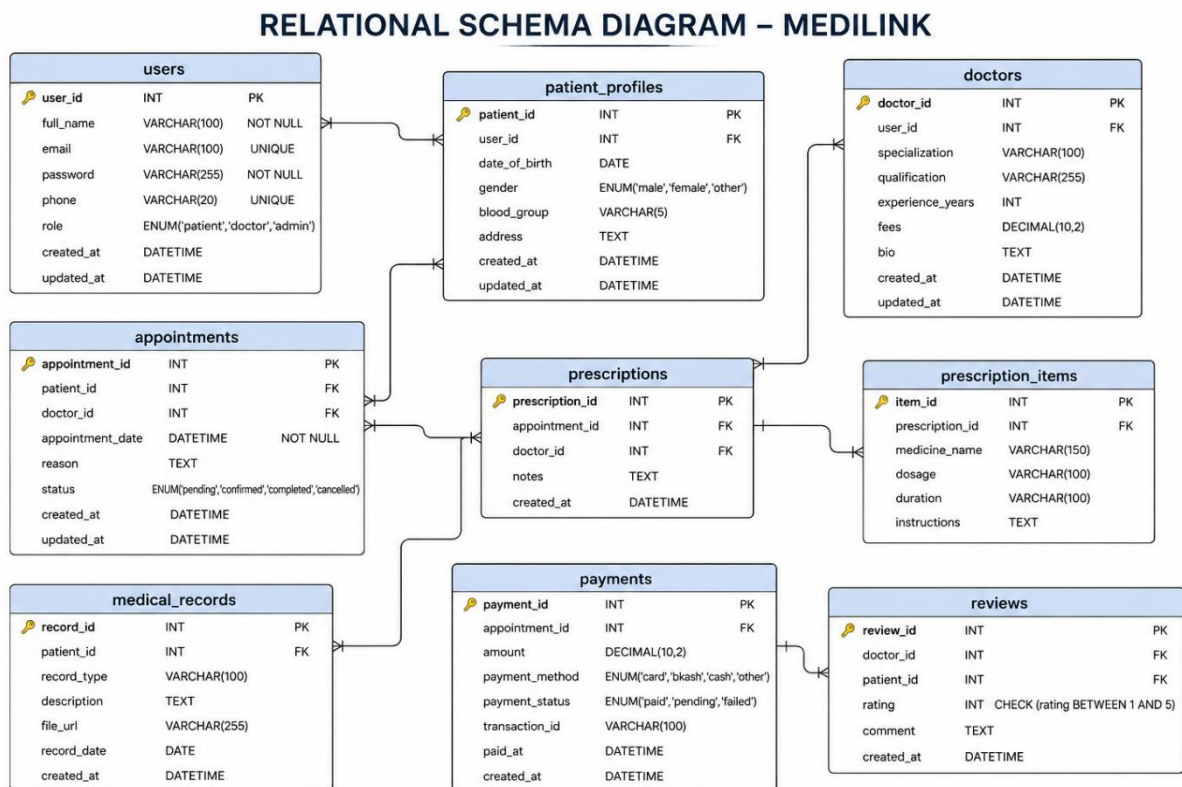


Description

The Entity Relationship (ER) Diagram of MediLink illustrates the major entities of the AI-Powered Smart Doctor Appointment and Healthcare Management System and the relationships among them. The primary entities in the system are Users, Patients, Doctors, Appointments, Prescriptions, Payments, Reviews, and Notifications. The Users entity stores general information about all system users, including name, email, password, phone number, role, and account creation date. A user can be registered as a patient, doctor, or administrator. The Patients entity contains patient-specific information such as date of birth, gender, blood group, and address. Similarly, the Doctors entity stores professional details including specialization, qualifications, experience, chamber address, consultation fees, and available working days. A patient can book multiple appointments with doctors through the Appointments entity. Each appointment contains appointment date, time, status, and the reason for consultation. After an appointment is completed, the doctor may generate a Prescription containing medications and treatment instructions. Appointment-related payments are recorded in the Payments entity,

which stores payment amount, method, status, and transaction details. Patients can provide feedback and ratings for doctors through the Reviews entity. In addition, the Notifications entity manages system-generated messages and alerts sent to users regarding appointments, prescriptions, and other healthcare activities. Overall, the ER Diagram provides a clear representation of how different entities interact within the MediLink healthcare ecosystem.

Relational Schema Diagram



Description

The Relational Schema Diagram represents the logical database structure of the MediLink system. It defines all tables, their attributes, primary keys, foreign keys, and data types.

The database consists of the following major tables:

- users
- patient_profiles
- doctors
- appointments

- prescriptions
- prescription_items
- medical_records
- payments
- reviews

Each table is designed to maintain data integrity and minimize redundancy through normalization. The users table serves as the central table that manages authentication and user information. The patient_profiles and doctors tables extend user information by storing patient-specific and doctor-specific details. The appointments table connects patients and doctors and stores appointment scheduling information. The prescriptions table records prescriptions generated after consultations, while prescription_items stores detailed medicine information such as dosage, duration, and instructions. The medical_records table maintains patient health records and uploaded documents. The payments table manages transaction details for appointments, and the reviews table stores patient feedback and doctor ratings. The schema establishes relationships through foreign keys, ensuring referential integrity and efficient database management.

Table Structure Examples

3. TABLE STRUCTURE EXAMPLES (Screenshot from Database)

users						
#	Name	Type	Null	Key	Default	Extra
1	user_id	int(11)	No	PRI	None	auto_increment
2	name	varchar(100)	No		None	
3	email	varchar(100)	No	UNI	None	
4	password	varchar(255)	No		None	
5	phone	varchar(15)	Yes		None	
6	role	enum('patient','doctor','admin')	No		None	
7	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

doctors						
#	Name	Type	Null	Key	Default	Extra
1	doctor_id	int(11)	No	PRI	None	auto_increment
2	user_id	int(11)	No	MUL	None	
3	specialization	varchar(100)	Yes		None	
4	qualification	varchar(150)	Yes		None	
5	experience	int(11)	Yes		None	
6	chamber_address	text	Yes		None	
7	fees	decimal(10,2)	Yes		None	
8	available_days	text	Yes		None	
9	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

appointments						
#	Name	Type	Null	Key	Default	Extra
1	appointment_id	int(11)	No	PRI	None	auto_increment
2	patient_id	int(11)	No	MUL	None	
3	doctor_id	int(11)	No	MUL	None	
4	appointment_date	date	No		None	
5	appointment_time	time	No		None	
6	status	enum('pending','confirmed','completed','cancelled')	No		None	
7	reason	text	Yes		None	
8	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

payments						
#	Name	Type	Null	Key	Default	Extra
1	payment_id	int(11)	No	PRI	None	auto_increment
2	appointment_id	int(11)	No	MUL	None	
3	amount	decimal(10,2)	No		None	
4	payment_method	varchar(50)	Yes		None	
5	payment_status	enum('pending','paid','failed')	No		None	
6	transaction_id	varchar(100)	Yes		None	
7	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

prescriptions						
#	Name	Type	Null	Key	Default	Extra
1	prescription_id	int(11)	No	PRI	None	auto_increment
2	appointment_id	int(11)	No	MUL	None	
3	medication	text	Yes		None	
4	instructions	text	Yes		None	
5	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

reviews						
#	Name	Type	Null	Key	Default	Extra
1	review_id	int(11)	No	PRI	None	auto_increment
2	patient_id	int(11)	No	MUL	None	
3	doctor_id	int(11)	No	MUL	None	
4	rating	int(11)	No		None	
5	comment	text	Yes		None	
6	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

notifications						
#	Name	Type	Null	Key	Default	Extra
1	notification_id	int(11)	No	PRI	None	auto_increment
2	user_id	int(11)	No	MUL	None	
3	message	text	Yes		None	
4	is_read	tinyint(1)	Yes		None	
5	created_at	timestamp	Yes		None	CURRENT_TIMESTAMP

Description

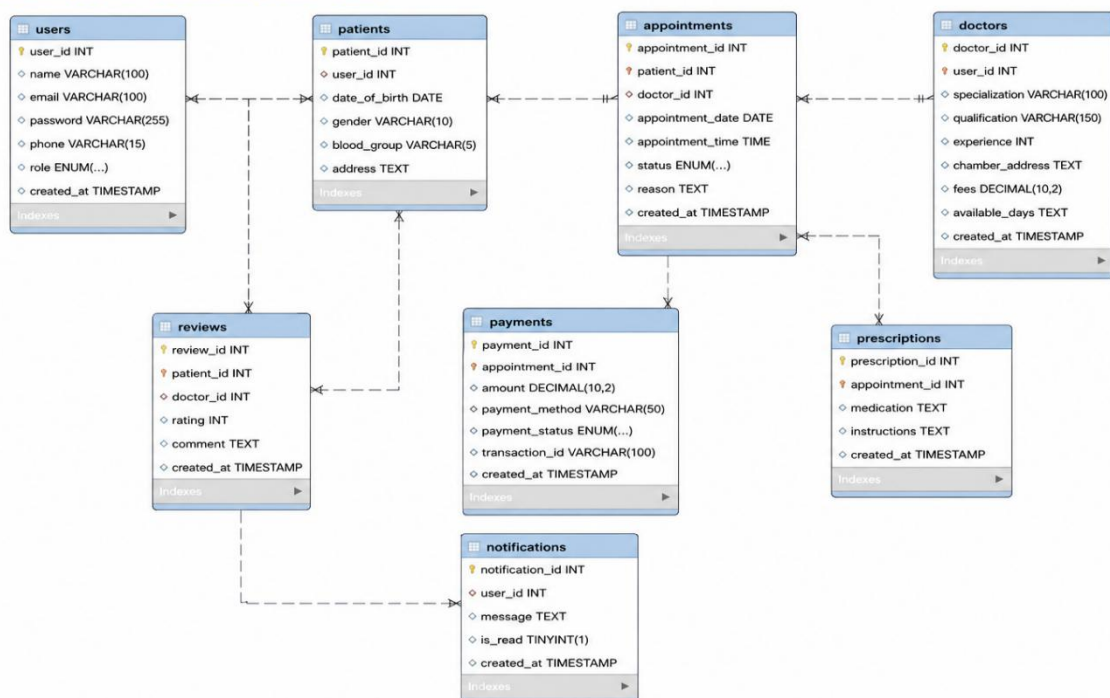
The Table Structure Examples section presents screenshots of actual database tables created in the MySQL database. The screenshots demonstrate the implementation of the system's database schema and display the following information:

- Column names
- Data types
- Primary keys
- Foreign keys
- Null constraints
- Default values
- Auto-increment properties

Examples include the Users, Doctors, Appointments, Payments, Prescriptions, Reviews, and Notifications tables. These screenshots verify that the database design has been successfully implemented according to the proposed schema and provide practical evidence of the database structure used in the MediLink system.

Relationship Diagram (DBMS View)

4. RELATIONSHIP DIAGRAM (DBMS VIEW)



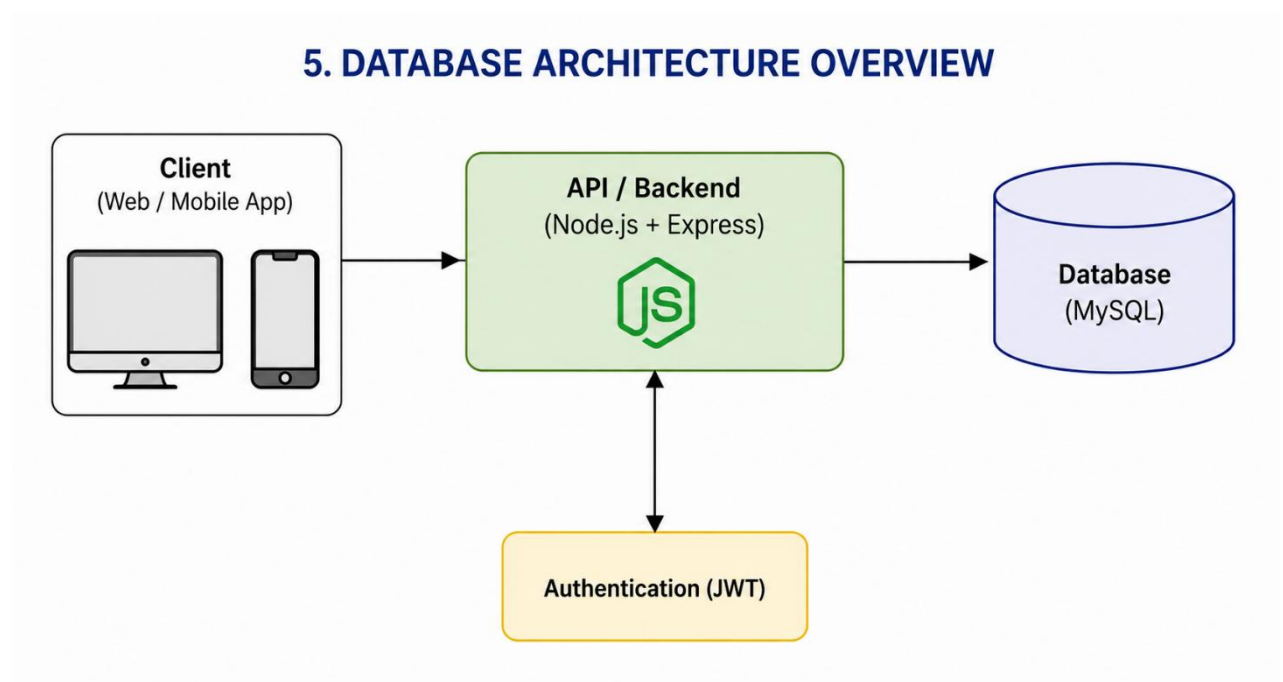
Description

The Relationship Diagram generated from the Database Management System (DBMS) visually represents the actual relationships among tables within the MySQL database. The diagram shows how primary keys and foreign keys connect various entities. For example:

- users is linked with patients and doctors.
- patients and doctors are connected through appointments.
- appointments are linked with prescriptions and payments.
- patients can submit reviews for doctors.
- users receive notifications generated by the system.

This DBMS-generated relationship diagram validates the correctness of the database design and confirms that all required relationships have been properly implemented in the database.

Database Architecture Overview



Description

The Database Architecture Overview illustrates the overall system architecture of MediLink. The architecture follows a three-tier structure:

1. Client Layer (Web and Mobile Applications)
2. API/Backend Layer (Node.js and Express.js)
3. Database Layer (MySQL)

Users interact with the system through web or mobile applications. Client requests are sent to the backend server through RESTful APIs. The backend processes business logic, authentication, appointment management, prescription generation, payment processing, and notification services. Authentication is handled using JSON Web Token (JWT), ensuring secure access control and session management. The backend communicates with the MySQL database to store, retrieve, and update healthcare-related information. This architecture provides scalability, security, maintainability, and efficient data management for the MediLink Healthcare Management System.